PISCINE MAKERS · EMBEDDING LAB

Semantic Agent Engine

Une mémoire qui sait ce qui a été décidé,
et ce qui est encore vrai.

Architecture v4 · Jonathan Chayna · octobre 2026

Architecture v4 — sept étapes nommées, du message qui arrive à la réponse citée. Chaque étape a une entrée, une sortie, un composant et une mesure.

L'idée en une minute

Une recherche classique retrouve les phrases qui se ressemblent, pas forcément celle qui est encore vraie. Ici, le message reste intact, le **chunker** en tire des phrases avec leurs offsets, le **tagger** lit nature, statut, point et valeur, le **linker** dit ce qui remplace quoi, la **memory** garde l'état courant et tout l'historique, le **retriever** mélange mots, sens et graphe, et l'**answerer** répond avec la phrase source — ou refuse.

Il comprend

Une proposition n'est pas une décision. Le statut est explicite : proposé, confirmé, annulé, réalisé.

Il se souvient

Message, phrase, auteur, date et preuve gardent leur identité. Rien n'est réécrit.

Il ne devine pas

Sans preuve : « je ne sais pas ». Aucun nom, nombre, date ou destinataire inventé.

Les sept étapes, en une ligne chacune

- **store** — le message entre tel quel ; la base refuse toute mise à jour
- **chunker** — phrases et unités, offsets exacts, confirmations courtes rattachées
- **tagger** — nature, statut, intention, point, valeur — Laya fine-tuné
- **linker** — même point, valeur différente, tour posé → lien supersedes
- **memory** — état append-only ; courant = dernier confirmé ; l'ancien reste
- **retriever** — BM25 + dense + graphe, fusion, Qwen3-Reranker-0.6B
- **answerer** — contexte + état → LLM ; réponse citée ou refus, chiffres vérifiés

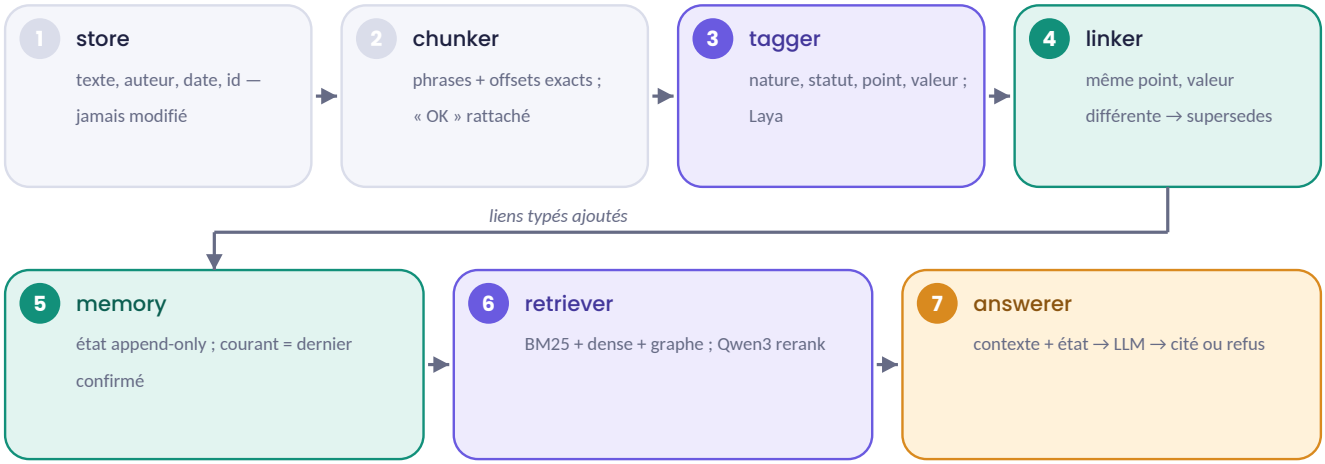
La règle de fer : le message et la phrase sont immuables ; les liens et les états sont ajoutés, jamais réécrits. L'état courant est une vue, pas une source de vérité.

Le chiffre

0,778 de réponses justes sur 5 réunions écrites à la main et 36 questions ; la bonne phrase est dans le top 10 dans **96 %** des cas ; **7** questions pièges sur 10 sont refusées. Chiffres régénérés par **make meetings**.

Le schéma

Sept composants nommés, un seul chemin : écrire à gauche, lire à droite



- **Ligne du haut** — ce qui arrive : le message est stocké, découpé, tagué, puis relié aux tours qui portaient l'ancienne valeur.
- **Ligne du bas** — ce qui se lit : la mémoire garde l'état courant, le retriever classe les tours, l'answerer écrit ou refuse. Le retriever ne modifie jamais la mémoire.
- **Le connecteur** entre linker et memory est le seul chemin d'écriture d'état : rien d'autre ne change l'état courant.

Couleurs : violet = partie entraînée, turquoise = déterministe ou mémoire, gris = sans entraînement, ambre = contrat et refus. Deux étapes n'ont volontairement aucun entraînement : le linker et la mémoire.

Le contrat, en clair

Un message ne change jamais ; une phrase garde ses offsets ; un remplacement est un lien ajouté ; un état est un événement ajouté ; une réponse cite une phrase ou refuse. C'est ce qui rend le système auditable : on peut toujours remonter de la réponse au texte exact.

Le chemin d'une décision

« on part sur 4 500, c'est acté » → **store** (t:17, 11/03) → **chunker** (1 unité) → **tagger** (point = prix, valeur = 4 500, confirmé) → **linker** (remplace les 4 800 du 03/03) → **memory** (courant = 4 500, les 4 800 restent) → **retriever** (top 1) → **answerer** (« 4 500, confirmé [traiteur:17:u0] »).

Le chemin d'une question sans état

« le traiteur c'est combien ? » → **retriever** (le tour du prix remonte) → aucun état rattaché à cette unité → **answerer** cite le tour directement (« 4 500 [traiteur:17:u0] »). C'est le cas qui échouait avant : un fait vrai sans état n'était pas répondable.

1 Sur l'exemple

Un message, deux morceaux, deux preuves

> *« Pour le projet X, on repousse le lancement à décembre. Au fait, où en est le back-end ? »*

- **Morceau A** — « on repousse le lancement à décembre » : décision. Le message reste intact ; un **StateEvent** met le lancement à décembre, l'ancienne valeur reste dans l'historique.
- **Morceau B** — « où en est le back-end ? » : question. On cherche après avoir enregistré A ; la réponse cite la phrase source.
- **Un « OK, validé » très court** reste attaché au message précédent ; il ne devient pas une phrase isolée.

2 2 · Chunker — découper sans perdre la preuve

Documents

On regroupe les phrases voisines qui parlent de la même chose. Cinq phrases sur un sujet : un seul morceau. Une phrase sur une voiture puis une sur un ballon : deux morceaux.

Conversations

On coupe dès que le sujet ou l'intention change, même au milieu d'une prise de parole. Un « OK, validé » court reste collé au message précédent.

Code

Une fonction = un morceau. Ce sont de simples règles de lecture du code, sans intelligence artificielle.

Toujours : même quand un morceau regroupe plusieurs phrases, chaque phrase d'origine garde ses offsets et reste la **preuve**. **text[start:end] == span.text** sur les 103 tours : 0 écart.

Cas	Règle	Exemple
deux phrases, même sujet	un seul morceau	budget proposé puis validé
le sujet ou l'intention change	on coupe	décision puis question
un « OK, validé » court	rattaché au message précédent	il ne devient pas une unité
du code	une fonction = un morceau	règles de lecture, sans IA

Le contrat immuable

Objet	Ce qui ne change jamais
Message	message_id, source, project, speaker, timestamp, text, hash
Chunk	chunk_id, message_id, start, end, text, sentence_index
Unit	unit_id, message_id, project, slot, point, value, status, kind, evidence
LinkEvent	ajouté : from, to, relation, confidence, evidence
StateEvent	ajouté : event (projet:slot:point), valeur, statut, evidence, date

Le **point** est le discriminateur qui manquait : sans lui, « clés », « fibre » et « bail » partageaient la même clé de date. La clé est **(projet, slot, point)**.

3 Les sept étapes

Chacune a un nom, une entrée, une sortie et une mesure

1. **store**
   Le message entre tel quel : identifiant, auteur, date, texte. La base refuse toute mise à jour ; ré-ingérer le même message ne fait rien. 0 doublon.
2. **chunker**
   Découpe en phrases et unités, avec offsets exacts. Les confirmations courtes restent rattachées. Sortie : des chunks qui pointent vers leur texte.
3. **tagger**
   Laya multilingue fine-tuné lit chaque unité : nature, statut, intention, point et valeur. Rappel des changements 1,00 ; 32 valeurs sur 32 sont des valeurs et pas des bouts de phrase.
4. **linker**
   Règle déterministe : même projet, même slot, même point, valeur différente, tour nouveau et posé → un lien **supersedes** vers le tour le plus récent qui portait l'ancienne valeur. 4 remplacements sur 4 reliés.
5. **memory**
   État append-only : une proposition reste ouverte, une confirmation met à jour la vue courante, l'ancien reste dans l'historique. 12 valeurs courantes sur 12 ; 0 fuite de statut entre unités.
6. **retriever**
   Entités et événements, puis BM25 + dense, expansion de graphe, fusion des classements et reranking Qwen3-Reranker-0.6B. hit@1 .577 · hit@3 .846 · hit@10 .962.
7. **answerer**
   La question, les tours retrouvés (id, date, auteur) et l'état courant partent au LLM ; la réponse n'est gardée que si elle cite un extrait fourni et que ses chiffres y figurent — sinon refus. Cite la bonne phrase .808 ; 7 pièges sur 10 refusés.

Ce que chaque étape reçoit et rend

Étape	Reçoit	Rend
store	message brut	ligne immuable
chunker	message	chunks + offsets
tagger	chunk	nature, point, valeur, statut
linker	unité + tours antérieurs	lien supersedes, ou rien
memory	liens + états	vue courante + historique
retriever	question	10 tours classés
answerer	question + tours + état	réponse citée ou refus

Ce qui change par rapport à v2/v3

- **v2** avait le bon parcours, mais « remplacer » pouvait réécrire l'ancien message.
- **v3** rendait la réunion immuable, mais le courant venait du dernier message, pas d'un état confirmé.
- **v4** nomme chaque étape, ajoute les offsets, le point, les liens typés et l'état append-only ; le courant est le dernier état confirmé, pas le dernier bavardage.

4 6 · Retriever — retrouver, puis lire l'état

Le cœur s'arrête ici : la réponse est une génération sur ce contexte

- **Interpréter la question** — Intention, slot demandé, personnes, dates ; résoudre « il », « elle », « ça », « le », « lui », « alors ».
- **Entités et événements** — Index séparés : personnes, projets, objets, décisions, actions, événements. « Le payer lui » retrouve le paiement et le prestataire.
- **BM25 + dense** — Mots exacts pour noms, dates, chiffres ; sens pour les reformulations ; fusion RRF.
- **Expansion de graphe** — Parcourir **same_event**, **answers**, **creates_action**, **supports**, **supersedes** pour retrouver les continuations distantes.
- **Reranking contextuel** — Qwen3-Reranker-0.6B relit la question avec chaque candidat ; le classement du reranker est fusionné avec celui du pipeline, pas substitué.

Le reranker, mesuré sur 26 questions répondables

Ordre	hit@1	hit@3	hit@5
ordre du pipeline	.346	.808	.846
reranker, consigne par défaut	.500	.692	.846
reranker, consigne française	.423	.885	.885
fusion + consigne française	**.577**	**.846**	.885

Le reranking seul gagne le top 1 et perd le top 3 ; la fusion garde les deux. Aucun fine-tune : 26 questions suffiraient à le faire gagner sur son propre jeu et à le perdre ailleurs.

Le choix de l'encodeur, mesuré

Encodeur	Ce qu'on a mesuré	Décision
e5-base	la référence sur les 36 questions	livré
jina v3	même niveau, parfois devant	rien à gagner
jina v5	même niveau que v3	rien à gagner
late chunking	gain net si la requête est contextualisée	gardé derrière --long-split

Ce que la recherche ne récupère pas encore

- Cinq questions répondables dont le point n'apparaît dans aucun tour : « un maintenant, un au troisième trimestre », « les 3 000 de différence », « la cause de l'incident », deux réponses anaphoriques « c'est moi ».
- Les embeddings ne changent rien de mesurable : jina v3 et v5 font jeu égal avec e5 ; e5-base reste livré pour la reproductibilité. Le gain venait du late chunking, pas du modèle.
- Le late chunking n'aide que si la requête est encodée avec son contexte : des vecteurs calculés à l'index avec une requête hors contexte inversent le signal (9/40).

5 4 · Linker et 5 · Memory

Décider ce qui remplace quoi, sans jamais réécrire

La règle du linker

- même **projet**, même **slot**, même **point**
- valeur **différente** et **tour nouveau**, déjà posé (pas une question, pas une hésitation)
- le lien pointe vers le tour le plus récent qui portait l'ancienne valeur
- rien n'est réécrit : « remplacé » est un lien, pas une modification

La clé d'état

(projet, slot, point) — le point distingue « clés », « fibre », « bail », « salaire », « acompte ».

Une proposition reste **ouverte** ; confirmé / annulé / réalisé met à jour la vue courante ; l'ancien reste dans l'historique.

Une unité ne reçoit un statut que si elle est la preuve de cet état : 76 fuites sur 103 unités avant la règle, **0** après.

Cinq clés réelles, telles que la mémoire les écrit

Projet	slot	point	Valeur	Statut
nova	date	lancement	5 avril	confirmé
hiring	budget	salaire	52 000	confirmé
traiteur	budget	acompte	1 350	proposé
demenagement	date	clés	1er septembre	confirmé
incident	budget	utilisateurs	12 000	confirmé

Qui devient « courant » ?

Unité	Statut	Courant ?	Pourquoi
« on pourrait remonter à 1 500 »	proposé	non	une proposition ne remplace pas une décision
« on valide 800 »	confirmé	oui	valeur confirmée, datée, avec preuve
« finalement on annule »	annulé	oui (annulé)	l'annulation est un état, pas un silence
« il a dit qu'il verrait »	inconnu	non	aucun engagement pris

Un état, trois messages

- **1er octobre** — « le budget est proposé à 1 200 € » : proposition, état ouvert.
- **8 octobre** — « on valide 800 € » : confirmation ; le courant devient 800 €, 1 200 € reste dans l'historique.
- **15 octobre** — « finalement on annule » : annulation ; le courant devient annulé, les deux valeurs restent visibles.
- **« On pourrait remonter à 1 500 € » ne remplace pas un 800 € confirmé.** Il reste ouvert.

Tout de suite

- découper, tagger, indexer le message
- chercher les candidats dans l'historique
- ajouter liens et état, sans réécrire

En arrière-plan

- recalculer les index d'entités et d'événements
- auditer le reranker et les seuils
- proposer un ré-entraînement sur données corrigées

6 Les résultats, étape par étape

5 réunions écrites à la main · 103 tours · 36 questions

Étape	Mesure	Résultat
1 · store	édition refusée ; ré-ingestion sans effet	refusée ; 0 doublon
2 · chunker	**text[start:end] == span.text**	0 écart sur 103 tours
3 · tagger	rappel des changements ; valeurs qui sont des valeurs	1,00 ; 32 sur 32
4 · linker	tour remplacé relié	4 sur 4
5 · memory	valeur courante ; proposition ouverte ; fuites	12 sur 12 ; 1 sur 1 ; 0
6 · retriever	hit@1 / hit@3 / hit@10	.577 / .846 / .962
7 · answerer	cite la bonne phrase ; pièges refusés	.808 ; 7 sur 10

Bout en bout : 0,778 — formel 0,833, désordonné 0,750, piège 0,700. Même jeu, trois rédacteurs :

Rédacteur	Réponses justes	Cite la bonne phrase	Pièges refusés
gabarit déterministe	.694	.731	9/10
LLM seul (7B)	.583	.577	7/10
hybride (livré)	**.778**	**.808**	7/10
LLM 4B	.278	0	10/10

Le 4B refuse tout : sur 8 Go, le 7B est le plancher utilisable. La mémoire répond quand elle a la preuve ; le LLM ne prend que le reste.

7 Ce qu'on entraîne

Chaque modèle a un rôle et une mesure ; deux étapes n'ont aucun entraînement

Étape	Modèle	Données	Mesure
3 · tagger	Laya multilingue, fine-tuné	12 500 / 2 000 / 4 200 exemples synthétiques, familles disjointes	rappel 1,00 ; 32/32
6 · retriever (sens)	e5-base gelé	paires réelles + synthétiques	hit@10 .962
6 · retriever (rerank)	Qwen3-Reranker-0.6B, zéro-shot	aucune	+.231 hit@1 par fusion
4 · linker	règle déterministe	aucune	4/4
5 · memory	règle append-only	aucune	12/12 ; 0 fuite
7 · answerer	Qwen2.5-7B-instruct + vérificateur	aucune	0,778 hybride

Le fine-tune du reranker a été écarté : 26 questions suffisent à le faire gagner sur son propre jeu et à le perdre ailleurs. Le tagger, lui, a 18 700 exemples et gagne pour de vrai.

8 Limites mesurées

Ce que les chiffres disent, sans les arrondir

- **Synthétique uniquement.** Cinq réunions, 36 questions, un or écrit par l'auteur des réunions. Aucune réunion tenue à l'écart : c'est un point de départ solide, pas un résultat de concours.
- **Les règles sont ajustées à ces cinq réunions** : statut, point, valeur. La prochaine preuve honnête est une réunion que les règles n'ont jamais vue.
- **hit@1 = .577.** La bonne phrase est dans le top 3 pour 85 % des questions, dans le top 10 pour 96 % — mais pas toujours première. Le reranker gagne le top 1 et le perd parfois au top 3 ; la fusion limite la casse.
- **3 questions pièges sur 10 reçoivent encore une réponse.** Le 7B conclut d'une intention qu'une action a eu lieu (« on va lui envoyer l'offre » → « il a accepté »). Un modèle plus grand est la variable manquante.
- **Deux clés pour un même point.** Un tour sans point écrit **projet:slot** à côté de **projet:slot:point**. Le linker devrait hériter du point quand il est unique.

Ce qui changerait le verdict

- une réunion tenue à l'écart, que ni les règles ni les seuils n'ont vue
- un answerer plus grand (14B ou hébergé) : les trois pièges restants sont une limite de modèle
- hériter du point quand il est unique, pour supprimer la double clé
- vingt réunions de plus pour séparer une règle qui marche d'une règle qui a de la chance

9 Petit lexique

Les mots techniques, en clair

store	Le message d'origine, tel quel : texte, auteur, date, identifiant. Jamais modifié.
chunker	Découpe le message en phrases et unités ; chaque phrase garde ses offsets exacts.
tagger	Lit chaque unité : nature, statut, point, valeur. C'est Laya, fine-tuné.
linker	Règle qui dit quel tour remplace quel autre. Elle ajoute un lien, elle ne modifie rien.
memory	L'état courant est une vue sur des événements ajoutés ; l'historique reste intact.
retriever	Mots exacts + sens + graphe, puis reranker : il trouve les tours candidats.
answerer	Question + tours retrouvés + état → LLM. La réponse cite ou refuse ; le vérificateur rejette tout chiffre absent des preuves.
abstention	Sans preuve suffisante, le moteur dit « je ne sais pas » au lieu d'inventer.

Tout se régénère : **make meetings** (bout en bout), **make steps** (tableau ci-dessus), **make ask4** (une question, à la main).